操作系统

课程导论

周炽金

华东师范大学
软件工程学院

任课教师简介

周炽金

主讲教师

联系方式

邮箱：
cjzhou@sei.ecnu.edu.cn
办公室：理科大楼 b1016

主要研究方向

Program analysis
AI for SE
AI system reliability

工作学习经历

2025.07—至今	华东师范大学 助理教授（晨晖学者）
2022.09—2025.06	清华大学　博士
2018.09—2021.06	清华大学　硕士
2014.09—2018.06	北京理工大学　学士

部分荣誉

清华大学优秀博士学位论文（Top 5%）
FSE 2022 杰出论文奖
USENIX Security 2024 杰出论文奖

课程简介

基本信息

《操作系统》
周三 9:50–12:15
教书院 316

《操作系统实践》
双周周二 14:50–16:25
开放教育学院 217

助教
颜晓轶、梁金哲
软件工程学院 2026 级硕士

教材
Operating System Concepts
10th Edition

其它优秀课程
MIT 6.S081
Operating System Engineering

北京大学・陈向群
《操作系统原理》

南京大学・蒋炎岩
《操作系统》

发布与交流
课件、公告：**学习通**
日常交流、答疑：微信群

群聊：2026年秋-操作系统（教书院316）



该二维码7天内（9月22日前）有效，重新进入将更新

考核方式

平时表现 40%

20% 日常学习
平时作业签到、课堂问答、前沿论文阅读

20% 期中考试
预计在第 10 周

期末考试 60%

根据卷面成绩评分

考试题型
选择题、判断题、应用题、综合题

或许能帮你提高分数的方法

- 概念很多，多问问大模型，加深理解。
- 多动手试一下，实验课不要懒。

你接触过哪些操作系统?


Windows 11


MacOS


iOS


android


Xiaomi HyperOS
小米澎湃OS


HarmonyOS NEXT


KYLIN
银河麒麟


统信UOS


ubuntu


archlinux


CentOS


OpenAnolis


OpenEuler


ROS
Robot Operating System

什么是操作系统?



经过这堂课，我们会知道的。

AI 时代下，为什么还要学习操作系统?

一道编程题

实现 `save_config(path, data)`，将新配置保存到文件。

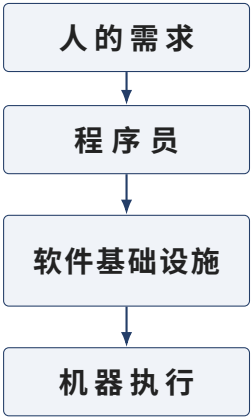
正常运行一次，重新打开文件，内容确实更新了。

- 写到一半，程序退出了，原配置还在吗?
- 已显示“保存成功”，机器断电后数据还在吗?
- 两个请求同时修改配置，会覆盖彼此的结果吗?

这些问题取决于**程序怎样执行，以及操作系统接口究竟保证什么**。

从需求到执行

程序员与软件基础设施，共同把人的需求落实为机器的执行。



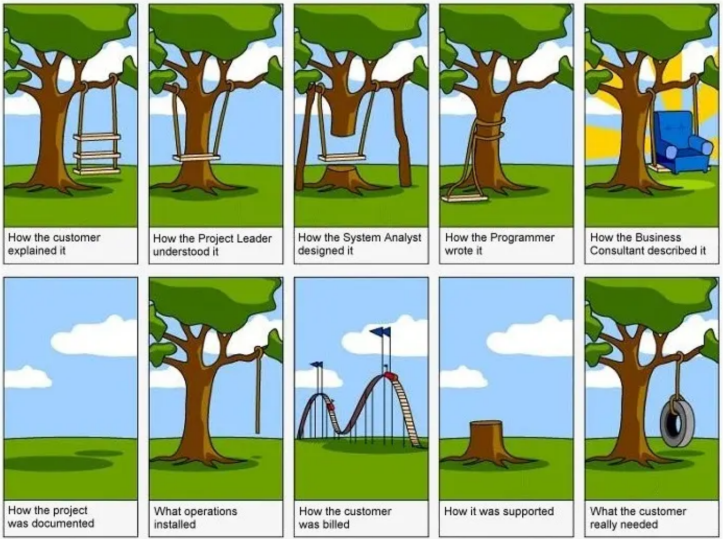
“下次打开应用时，记住我的设置。”

明确保存什么、何时保存、失败后怎么办，
编写保存程序。

编译器或解释器：翻译、执行程序。
运行库和操作系统：提供文件、内存等支持。

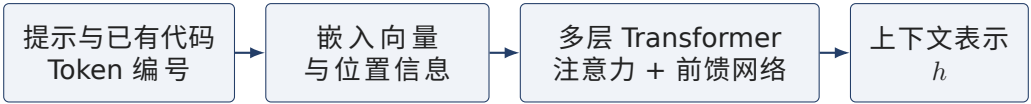
执行指令，改变内存与存储中的状态。

需求的不同理解


How the customer explained it
How the Project Leader understood it
How the System Analyst designed it
How the Programmer wrote it
How the Business Consultant described it
How the project was documented
What operations installed
How the customer was billed
How it was supported
What the customer really needed

LLM 是怎么工作的?

以基于 Transformer 的自回归语言模型为例:



Token 与嵌入
文本分成词元，编号后查表得到向量；
位置信息帮助区分顺序。

自注意力：匹配，再加权
由向量算出 Q（查询）、K（匹配特征）、
V（待组合信息）。Q 与 K 匹配，
经 softmax 得到权重，再对 V 加权。

$$z_i = \sum_{j \le i} \alpha_{ij} v_j, \qquad \sum_{j \le i} \alpha_{ij} = 1$$

当前位置只使用自身及前文。
多头注意力从不同投影组合信息。

每层还有**前馈网络、残差连接与归一化**；
多层计算后，最后位置的表示 h
用于预测下一个 token。

资料：[1] Vaswani et al., 2017　[2–4] D2L Transformer 相关章节

语言模型在预测什么?

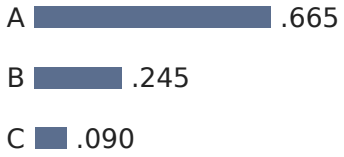
上下文表示 h → 词表分数 → softmax → 条件概率分布

$$p_\theta(x_{t+1} = v \mid c, x_{1:t}) = \mathrm{softmax}(W_{\mathrm{out}} h + b)_v$$

c：提示、要求与已提供的资料；$x_{1:t}$：已生成的 token；θ：学到的参数。

每次生成一步

词表 A、B、C 的分数为 $2, 1, 0$，
右侧为 softmax 归一化后的概率。



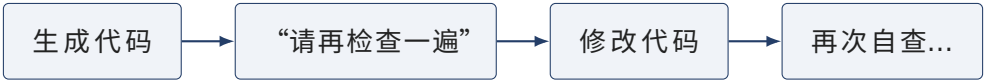
选择一个 token，追加到上下文，
再预测下一步，逐步生成整段代码。

常见预训练通过提高训练文本中实际后续 token 的概率来学习参数。

概率模型无法提供正确性保证

资料：[1] Vaswani et al., 2017　[5] Brown et al., 2020

反复自查是否能改对?



普通对话中的自查会增加上下文，通常不会更新模型参数，
也不自动带来执行证据。

仍以保存配置为例

如果始终只检查“正常保存后能否读回”，
反复修改仍可能漏掉“写到一半退出”。

模型可能发现错误，也可能重复遗漏，甚至把正确部分改错；
迭代次数本身不保证质量提升。

研究边界：Huang 等观察到无外部反馈的纠错可能退步

资料：[7] Huang et al., ICLR 2024

使用 AI 编程时，我们还需要做什么？

- **把程序的要求说清楚。**
  保存配置，是正常写入就够了，还是中途失败、断电、同时修改时也有要求？
- **看懂 AI 代码调用的操作系统接口。**
  关闭文件、替换文件分别做了什么？它们能保证什么？
- **检查代码没有考虑到的执行情况。**
  正常运行通过后，再检查写到一半退出、保存后断电、两个请求同时修改。
- **根据实际执行结果，指出错误并检查修改。**
  可以让 AI 生成测试和修改代码；我们仍要核对测试是否覆盖要求、修改是否解决问题。

配置文件保存

常见写法，以下为教学示例：

```
def save_config(path, data):
    with open(path, "w", encoding="utf-8") as f:
        f.write(data)
```

正常测试：保存后重新读取，确实得到完整新配置。

`with` 会关闭文件。还需要区分：

- 正常结束时，文件内容更新。
- 失败以后，仍有一份**完整配置**可用。

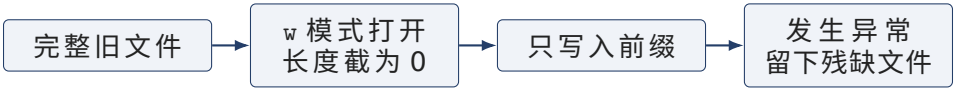
这两个要求，对实现的要求不同。

资料：[9] Python 文档

写到一半失败了怎么办?

旧：`{"theme":"light","font":12}`

新：`{"theme":"dark","font":14}`



示例

写入前 12 字节、刷新缓冲，再抛异常。重开文件：

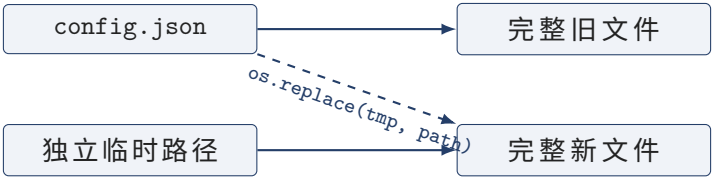
```
{"theme":"da
```

旧配置已被破坏，新配置还未写完。

资料：[9] Python 文档

先写临时文件，再替换

先写同目录下独立的临时文件，写完整后再替换目标路径。



替换前失败：目标仍是旧文件；替换成功后：新打开的读者得到新文件。
已经打开旧文件的读者，可以继续读取旧对象。

原子路径替换解决了可见性问题；断电和并发事务仍需单独讨论。

资料：[10] Linux man-pages　[12] Python 文档

保存成功后，断电是否还会丢？



`close` 或路径替换成功，不自动等于断电后数据仍在。

要求：成功返回后，断电仍保留新配置

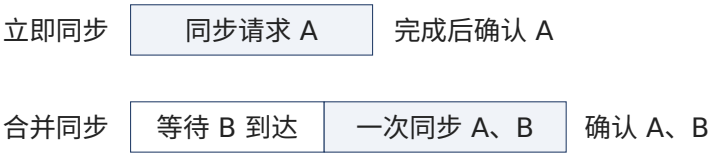
写临时文件 → `flush` → `fsync`(临时文件)
→ 替换路径 → `fsync`(父目录) → 返回成功

文件数据与**目录项**是不同的持久化对象。
每一步都要检查错误，并明确文件系统与设备的保证。

资料：[11] Linux man-pages [12] Python 文档 [13] OSTEP 第 42 章

持久化与性能

上一页要求保存成功前完成持久化；等待同步会增加响应时间。
以同一追加日志上的请求为例：



只表示操作顺序，不代表实测时长；合并对象为同一日志。

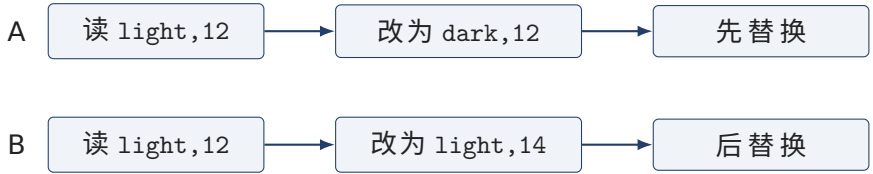
合并可减少同步次数，较早到达的请求要等待收集时间。

如果在同步**之前**确认“持久保存成功”，改变的是承诺本身。

资料：[11] Linux man-pages　[13] OSTEP 第 42 章

两个请求同时修改会怎样?

初始配置：`theme=light, font=12`



最终为 `light,14`：A 对 theme 的修改丢失。

原子替换不等于**读取—修改—保存**整体原子。
需要定义冲突策略：整体加锁、版本检查后提交，或按语义合并。

资料：[10] Linux man-pages　[14] OSTEP 第 26 章

保存功能的测试

部分写入

写入前缀后抛异常，再重开文件；检查是否仍有完整版本。

两个请求同时修改会怎样?

让两个请求先读同一旧版本，再依次提交；检查冲突策略。

崩溃恢复

在写入、同步、替换之间注入崩溃；检查恢复状态与已确认操作。
必须明确故障模型：杀死应用进程，不等于整机断电。

测试通过说明**这些执行**符合要求；仍需分析未覆盖的执行与假设。

资料：[13] OSTEP 第 42 章　[14] OSTEP 第 26 章

同一个 `save_config`，放在不同场景中，合理实现可能不同。

可重建缓存

允许丢失最近结果，未必需要每次等待持久化。

用户确认的配置变更

若承诺断电后保留，就需要相应的同步与恢复语义。

多人共同修改

还要定义冲突、协调提交，避免不符合要求的覆盖。

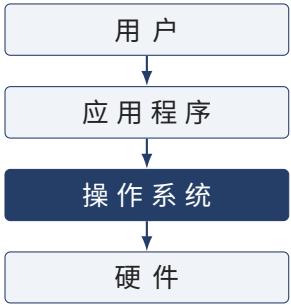
理解文件、缓存和并发的执行机制，才能判断 AI 的方案
满足了哪些条件、遗漏了哪些条件，以及代价是否值得。

什么是操作系统?

An operating system is a program that **manages** a computer's **hardware**.

It also provides a **basis for application programs**

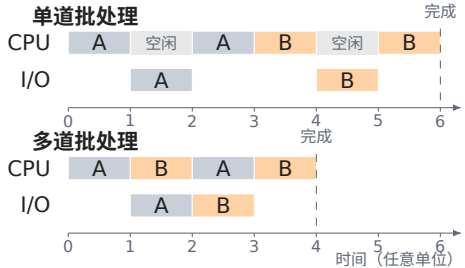
and acts as an **intermediary** between the **computer user** and the **computer hardware**.



资料：[15] Operating System Concepts，第 1 章

从人工操作到分时

- **早期：人工操作**
  人工装入程序，更换作业需要等待。
- **1950 年代：单道批处理**
  作业自动依次执行；一个完成后才运行下一个，等待 I/O 时 CPU 空闲。
- **1960 年代：多道批处理**
  多个作业驻留内存；一个等待 I/O 时，CPU 可以运行另一个。
- **1960 年代：分时**
  在用户任务间快速切换，让多人通过终端交互使用。





1966 年的终端教室
UC Santa Barbara
IBM 档案

资料：[16] OSTEP §2.6　[18] CHM：早期操作系统　[19] IBM：Time-sharing

从 UNIX 到移动操作系统

- **1970 年代：UNIX 与可移植性**
  1973 年用 C 重写内核，便于修改和移植到不同机器。
- **1980 年代：个人电脑与图形界面**
  电脑走向个人；Macintosh（1984）用窗口、图标和鼠标交互。
- **1990 年代：Linux 与开源协作**
  Linux 内核于 1991 年发布，开发者共同完善系统。
- **2000 年代以来：智能手机普及**
  手机上的操作系统需要兼顾交互体验、续航与安全。



Macintosh，1984
Computer History Museum

资料：[20] Dennis Ritchie：UNIX 历史 [21] CHM：1984 / 1991 [15] Operating System Concepts，第 1 章

程序运行需要哪些资源?

以编辑一份文档为例:

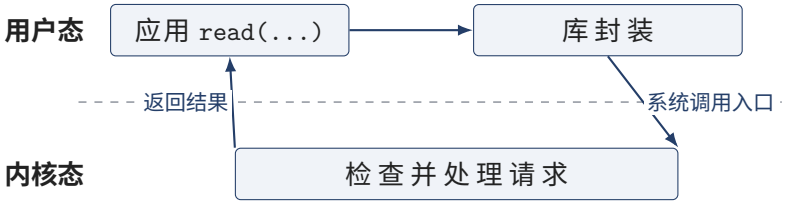
资源	作用	例子
CPU	执行指令	计算排版、处理输入
内存	存放正在使用的指令和数据	文档内容、撤销记录
持久存储	长期保存数据	SSD 上的文档文件
输入输出设备	与外界交换信息	键盘输入、屏幕显示

- 内存与持久存储用途不同:编辑时的数据,不一定已经保存到文件中。
- 多个应用共享这些资源;CPU 计算与设备工作也可以重叠进行。

资料:[15] Operating System Concepts,第 1 章

程序怎样使用操作系统?

以读取文件为例，应用提出读请求，内核处理后返回结果。



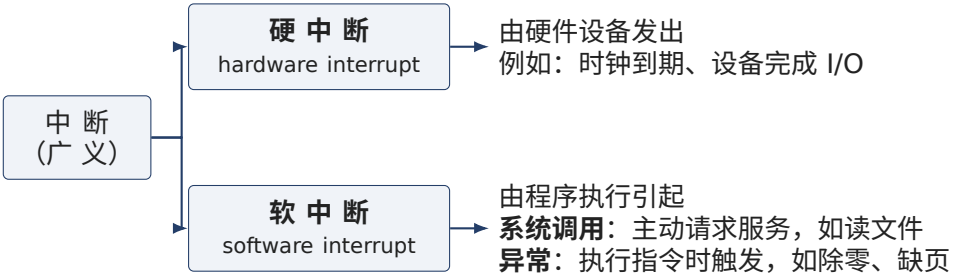
- **系统调用**：应用请求内核服务的入口，例如文件读写、创建进程。
- **用户态与内核态**：执行权限不同；普通应用不能随意执行特权操作。
- 应用调用的普通函数，不一定触发系统调用。

资料：[15] Operating System Concepts，第 1–2 章

操作系统何时获得控制?

An operating system is **interrupt driven**.

当 CPU 正在执行应用时，**中断使控制转入内核**。



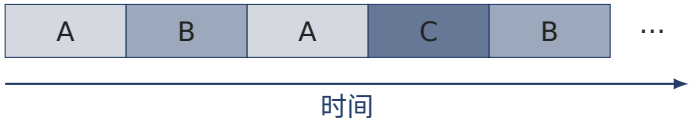
例如：时钟中断 → 内核获得控制 → 决定哪个任务继续运行。

资料：[15] Operating System Concepts，第 1 章：Common Functions of Interrupts / Operating-System Operations

多个程序怎样共享 CPU?

- **进程**：一个程序运行时的实例，包含运行状态和所使用的资源。
- **线程**：进程内的一条执行流；同一进程可以包含多个线程。
- **调度**：决定哪个可运行的线程获得 CPU、何时切换。

以单核、单硬件线程为例：



多个任务轮流执行，形成**并发**；多个核心还可以让任务**并行**执行。

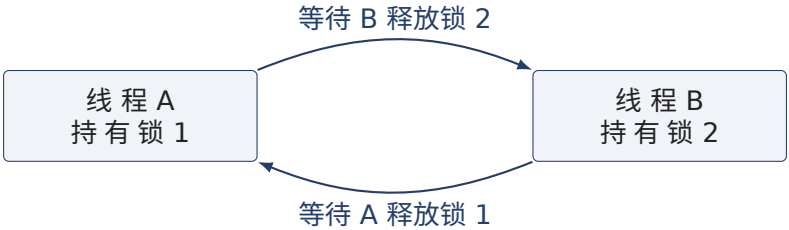
资料：[15] Operating System Concepts，第 1、3–5 章　[16] OSTEP，第 2 章

并发程序如何协作?

前面的两个配置请求会覆盖彼此的修改，共享访问需要协调。

- **同步**：用互斥、等待与通知等机制，协调共享数据的访问和执行顺序。
- **死锁**：多个执行者相互等待对方占用的资源，无法继续。

例如，两个线程按相反顺序获取两把锁：

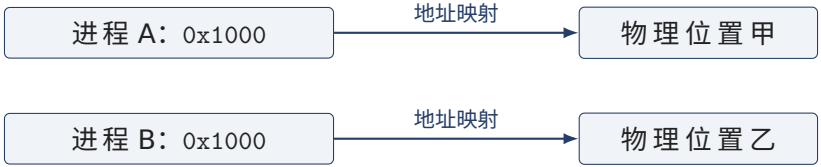


加锁可以保护共享数据，锁的使用方式也需要设计。

资料：[15] Operating System Concepts，第 6–8 章

程序的内存如何管理?

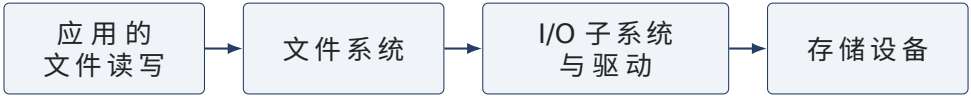
程序看到的地址，与内存中的实际位置可以不同。



- **分配与回收**：给程序提供内存，并在不再使用时收回。
- **地址映射与隔离**：让程序使用各自的地址空间，限制相互访问。
- **虚拟内存**：按需将内容放入内存；内存紧张时，可以置换部分内容。

资料：[15] Operating System Concepts，第 9–10 章

文件读写如何实现?



- **文件系统**：组织文件与目录，管理文件数据在存储上的位置。
- **存储管理**：管理可用空间、分配存储位置，安排设备请求。
- **I/O 子系统与驱动**：把请求交给具体设备，处理数据传输与完成通知。

应用使用文件名和读写接口，不必自行操作磁盘扇区。

资料：[15] Operating System Concepts，第 1、11–15 章

资源如何受到保护?

- 一个程序不能随意改写另一个程序的数据。
  地址空间与访问权限限制内存访问。
- 普通应用不能任意控制硬件或修改内核。
  用户态、内核态与受控入口限制特权操作。
- 用户不能读取自己无权访问的文件。
  身份与访问控制决定谁可以读、写或执行。

保护规定“谁能访问什么”；安全还包括防范漏洞利用和其他攻击。

资料：[15] Operating System Concepts，第 1、16–17 章

这门课学什么?

- **程序怎样获得系统服务?**
  操作系统服务、系统调用、系统结构。
- **多个任务怎样运行?**
  进程、线程、CPU 调度。
- **共享资源怎样协调?**
  同步工具、同步问题、死锁。
- **程序怎样使用内存?**
  主存管理、地址映射、虚拟内存。
- **数据怎样保存和传输?**
  存储管理、I/O、文件系统接口与实现。

保护与安全贯穿这些主题。

在实践中理解操作系统

理论课解释机制，实践课通过代码观察和修改这些机制。

- **使用 Linux 开发与调试程序**
  编译、运行，观察进程与线程，跟踪执行。
- **阅读 Pintos 代码**
  了解一个教学操作系统如何组织、编译和运行。
- **完善已有功能**
  忙等待与优先级调度、参数传递与系统调用。
- **解释和验证修改**
  结合执行过程说明问题，用测试检查实现是否满足要求。

Why do we disallow LLMs?

- It is easier than ever to use LLMs to write code
- But then – why should anyone hire YOU if it is so easy???
- What is your incremental value?
- LLMs still hallucinate – they are statistical entities!
  - Lots of buggy code being created by clueless people
  - Example: people writing apps for phones that look good but leak private information
  - Worse example: writing code at core of systems that have synchronization bugs
- Your incremental value:
  - *Eventually* using LLMs effectively to write good code
  - Figure out when they are wrong and correct them
- You need to be an expert to do this
- BEING AN EXPERT STARTS WITH UNDERSTANDING
  - Can't do that if you just turn in AI-generated code
  - We will be testing your understanding by asking you to explain and demonstrate

资料：[17] John Kubiatowicz —UC Berkeley CS162, Fall 2026, Lecture 1, p. 52

AI 编程与课程考核

- **允许使用 AI 辅助编程**
  成绩取决于你对代码和原理的理解。

- **实践课：更看重展示与问答**
  讲清设计、关键代码和测试依据，并回答现场追问。

- **理论课：通过考试检验理解**
  解释机制，分析执行过程，解决具体问题。

例如：**这把锁保护了什么？去掉它会发生什么？**

参考资料

语言模型

[1] Vaswani et al., 2017 —Attention Is All You Need

[2] D2L §11.6 —Self-Attention and Positional Encoding

[3] D2L §11.3 —Attention Scoring Functions

[4] D2L §11.7 —The Transformer Architecture

[5] Brown et al., 2020 —Language Models are Few-Shot Learners

[6] DeepSeek-AI, 2025 —DeepSeek-R1

[7] Huang et al., ICLR 2024 —Large Language Models Cannot Self-Correct Reasoning Yet

[8] Kumar et al., 2024 —Training Language Models to Self-Correct via Reinforcement Learning

历史资料

[18] CHM：早期操作系统；**[19]** IBM：Time-sharing。

[20] Dennis Ritchie：UNIX 历史；21. CHM：1984 / 1991。

文件与并发

[9] Python 文档—open()

[10] Linux man-pages —rename(2)

[11] Linux man-pages —fsync(2)

[12] Python 文档—os.fsync() / os.replace()

[13] OSTEP 第 42 章—Crash Consistency: FSCK and Journaling

[14] OSTEP 第 26 章—Concurrency and Threads

课程概览

[15] Silberschatz、Galvin、Gagne —Operating System Concepts，第 10 版配套课件。

[16] Arpaci-Dusseau、Arpaci-Dusseau —OSTEP，第 2 章：Introduction to Operating Systems。CPU、内存、并发与持久性的直观介绍。

[17] John Kubiatowicz —UC Berkeley CS162, Fall 2026, Lecture 1, p. 52：Why do we disallow LLMs?（教师提供的原图）。